

Guía Semana 6: Módulos Nativos y Sensores del Dispositivo

Sep 28, 2026 · @AntonioJGL

Esta semana construirás GeoCam, una cámara que etiqueta cada foto con coordenadas GPS y reacciona al movimiento del teléfono, encapsulando el hardware en Custom Hooks tipados.

Objetivos y preparación

Al terminar esta guía podrás:

1. Explicar cómo JavaScript llega al hardware a través de los módulos nativos de Expo y cuándo Expo Go deja de ser suficiente.
2. Implementar el ciclo completo de permisos: verificar, explicar, solicitar, degradar y redirigir a Ajustes.
3. Encapsular cámara, ubicación y sensores en Custom Hooks con limpieza correcta de suscripciones.
4. Auditar código generado por IA para detectar APIs obsoletas de Expo.

Antes de clase

- ☐ Tener Expo Go instalado en un teléfono físico (la cámara no funciona en el simulador de iOS).
- ☐ Tener clonado el repositorio del Taller Integrador 2.
- ☐ Conectar el teléfono a la misma red que el computador, o usar `npx expo start --tunnel`.
- ☐ Leer la documentación de Expo Camera y Expo Location (enlaces al final).

Preguntas de repaso

Intenta responderlas antes de clase; las discutiremos al inicio.

1. ¿Qué diferencia hay entre un permiso `denied` y uno con `canAskAgain: false`?
2. ¿Por qué un `useEffect` que se suscribe a un sensor necesita función de limpieza?
3. ¿Dónde se declaran los textos que iOS muestra al pedir un permiso?
4. Si una librería nativa no está incluida en Expo Go, ¿qué alternativa existe?

Conceptos clave

La idea central: el hardware se usa a través de módulos nativos, siempre con permiso explícito del usuario y siempre liberando lo que se suscribe.

Cómo llega JavaScript al hardware

Cuando llamas a `Location.getCurrentPositionAsync()`, TypeScript no toca el GPS. En la New Architecture la llamada pasa por JSI hacia código nativo en Kotlin (Android) o Swift (iOS). Las librerías `expo-*` están construidas sobre la Expo Modules API, que da una interfaz uniforme a esos módulos.

Expo Go es una app ya compilada con un conjunto fijo de módulos nativos. `expo-camera`, `expo-location`, `expo-sensors`, `expo-image-picker` y `react-native-maps` ya vienen incluidos. Una librería nativa que Expo Go no trae requiere un development build (`expo-dev-client` + EAS Build, Semana 14).

Los config plugins de `app.json` modifican automáticamente `Info.plist` y `AndroidManifest.xml` al compilar, sin tocar código nativo.

Los textos de permisos definidos en `app.json` solo se ven en builds propios. En Expo Go verás los textos genéricos de Expo Go; no es un error de tu código.

El modelo de permisos

Aspecto	iOS	Android
Texto de justificación	Obligatorio (<code>NSCameraUsageDescription</code> , etc.); sin él la app se cierra	Opcional; el sistema muestra su propio diálogo
Oportunidades de preguntar	Una sola; tras rechazar, solo queda Ajustes	Desde Android 11, tras dos rechazos el diálogo deja de aparecer
Ubicación aproximada	Desde iOS 14	Desde Android 12

Expo unifica ambas plataformas en una respuesta con tres campos:

```
interface PermissionResponse {
  status: 'granted' | 'denied' | 'undetermined';
  granted: boolean;
  canAskAgain: boolean; // false = hay que enviar al usuario a Ajustes
}
```

De ahí sale la máquina de estados que usarás en todos los hooks:

```
checking -> undetermined -> (acepta) -> granted
                                     -> (rechaza) -> denied -> (rechaza de nuevo) ->
blocked
blocked -> Linking.openSettings() -> (lo activa en Ajustes) -> granted
```

Buenas prácticas de UX de autorización:

1. **Pedir en contexto.** La cámara se pide al tocar "Tomar foto", no al abrir la app.
2. **Pantalla previa de explicación.** Como iOS da una sola oportunidad, explicar antes del diálogo del sistema aumenta la aceptación.
3. **Degradar con elegancia.** Sin ubicación, la cámara sigue funcionando; la foto queda sin coordenadas.
4. **Manejar el estado bloqueado.** Ofrecer "Abrir Ajustes" en lugar de un botón que no hace nada.
5. **Pedir solo lo necesario.** Ubicación en primer plano basta; la de segundo plano exige justificación ante las tiendas (Semana 12).

Cámara, galería, ubicación y sensores

Módulo	API clave	Detalle que causa errores
<code>expo-camera</code>	<code>CameraView</code> , <code>useCameraPermissions()</code> , <code>ref.takePictureAsync()</code>	Esperar <code>onCameraReady</code> ; <code>CameraView</code> no admite hijos, los controles van como hermanos absolutos
<code>expo-image-picker</code>	<code>launchImageLibraryAsync({ mediaTypes: ['images'] })</code>	El selector del sistema moderno no requiere pedir permiso de galería para elegir una imagen
<code>expo-location</code>	<code>getForegroundPermissionsAsync</code> , <code>getCurrentPositionAsync</code> , <code>watchPositionAsync</code>	La suscripción de <code>watchPositionAsync</code> debe eliminarse con <code>.remove()</code>

Módulo	API clave	Detalle que causa errores
react-native-maps	MapView, Marker	Funciona en Expo Go; en producción Android requiere API key de Google Maps
expo-sensors	isAvailableAsync, setInterval, addListener	Toda suscripción se limpia; la frecuencia afecta la batería

Precisión de ubicación: `Accuracy.Balanced` sirve para etiquetar fotos; `BestForNavigation` es para apps de navegación.

Sensor	Mide	Unidad	Uso típico
Accelerometer	Aceleración en x, y, z	g (en reposo la magnitud es cercana a 1)	Detectar agitado, pasos, caídas
Gyroscope	Velocidad de rotación	rad/s	Juegos, gestos de giro
Magnetometer	Campo magnético	μT	Brújula
Barometer	Presión atmosférica	hPa	Altitud relativa
Pedometer	Pasos	conteo	Apps de salud

Un intervalo de 100 ms basta para detectar un agitado; 16 ms (60 Hz) solo se justifica en juegos.

La regla de oro: toda suscripción se limpia

Ejercicio: encuentra los dos errores de este código, típico de la IA, antes de leer la respuesta.

```
// Incorrecto
useEffect(() => {
  Location.watchPositionAsync({}, (loc) => setCoords(loc.coords));
}, []);
```

1. La suscripción nunca se elimina: el GPS sigue encendido al salir de la pantalla y consume batería.

2. Como `watchPositionAsync` es asíncrono, si el componente se desmonta antes de que la promesa se resuelva, la suscripción queda huérfana. La solución es una bandera `cancelled`, que verás en `useGeolocation`.

Práctica guiada: GeoCam paso a paso

Construirás dos hooks, un componente de permisos y la pantalla GeoCam, probando cada estado de permiso en tu teléfono.

Paso 1. Instalación y configuración

```
npx expo install expo-camera expo-location expo-image-picker expo-sensors
react-native-maps
```

Usa `npx expo install` y no `npm install`: Expo resuelve la versión compatible con el SDK de tu proyecto.

Agrega los plugins en `app.json`:

```
{
  "expo": {
    "plugins": [
      ["expo-camera", {
        "cameraPermission": "GeoCam usa la cámara para tomar fotos
geolocalizadas.",
        "recordAudioAndroid": false
      }],
      ["expo-location", {
        "locationWhenInUsePermission": "GeoCam usa tu ubicación para
registrar dónde tomaste cada foto."
      }],
      ["expo-image-picker", {
        "photosPermission": "GeoCam accede a tu galería para importar fotos."
      }]
    ]
  }
}
```

Estructura de carpetas objetivo:

```

app/(tabs)/_layout.tsx
app/(tabs)/geocam.tsx
app/(tabs)/mapa.tsx
components/PermissionPrimer.tsx
hooks/useGeoLocation.ts
hooks/useCamera.ts
hooks/useShake.ts
context/GeoPhotosContext.tsx
types/geo.ts

```

Paso 2. Tipos compartidos

```

// types/geo.ts
export type PermissionState =
  | 'checking'
  | 'undetermined'
  | 'granted'
  | 'denied'
  | 'blocked';

export interface Coords {
  latitude: number;
  longitude: number;
  accuracy: number | null;
  timestamp: number;
}

export interface GeoPhoto {
  id: string;
  uri: string;
  coords: Coords | null; // null si el usuario negó la ubicación
  source: 'camera' | 'gallery';
  createdAt: number;
}

```

`coords: Coords | null` lleva al sistema de tipos la regla de degradar con elegancia: TypeScript te obliga a manejar la foto sin ubicación.

Paso 3. Custom Hook useGeoLocation

```

// hooks/useGeoLocation.ts
import { useCallback, useEffect, useState } from 'react';
import { Linking } from 'react-native';
import * as Location from 'expo-location';
import type { Coords, PermissionState } from '@types/geo';

```

```

interface Options {
  watch?: boolean;
}

interface GeoLocationState {
  permission: PermissionState;
  coords: Coords | null;
  error: string | null;
}

function mapPermission(res: Location.LocationPermissionResponse):
PermissionState {
  if (res.granted) return 'granted';
  if (!res.canAskAgain) return 'blocked';
  if (res.status === 'undetermined') return 'undetermined';
  return 'denied';
}

function toCoords(loc: Location.LocationObject): Coords {
  return {
    latitude: loc.coords.latitude,
    longitude: loc.coords.longitude,
    accuracy: loc.coords.accuracy,
    timestamp: loc.timestamp,
  };
}

export function useGeoLocation({ watch = false }: Options = {}) {
  const [state, setState] = useState<GeoLocationState>({
    permission: 'checking',
    coords: null,
    error: null,
  });

  // 1. Al montar: solo CONSULTAR el permiso, nunca pedirlo
  useEffect(() => {
    let cancelled = false;
    Location.getForegroundPermissionsAsync()
      .then((res) => {
        if (!cancelled) setState((s) => ({ ...s, permission:
mapPermission(res) }));
      })
      .catch(() => {
        if (!cancelled) setState((s) => ({ ...s, permission: 'denied' }));
      });
  });
}

```

```

    return () => {
      cancelled = true;
    };
  }, []);

// 2. Solicitar permiso por acción explícita del usuario
const requestPermission = useCallback(async (): Promise<boolean> => {
  const res = await Location.requestForegroundPermissionsAsync();
  setState((s) => ({ ...s, permission: mapPermission(res) }));
  return res.granted;
}, []);

// 3. Lectura única, útil justo antes de tomar la foto
const getCurrent = useCallback(async (): Promise<Coords | null> => {
  try {
    const loc = await Location.getCurrentPositionAsync({
      accuracy: Location.Accuracy.Balanced,
    });
    const coords = toCoords(loc);
    setState((s) => ({ ...s, coords, error: null }));
    return coords;
  } catch (e) {
    const message = e instanceof Error ? e.message : 'No se pudo obtener la
ubicación';
    setState((s) => ({ ...s, error: message }));
    return null;
  }
}, []);

// 4. Seguimiento continuo con limpieza segura ante desmontaje temprano
useEffect(() => {
  if (!watch || state.permission !== 'granted') return;

  let cancelled = false;
  let subscription: Location.LocationSubscription | null = null;

  Location.watchPositionAsync(
    { accuracy: Location.Accuracy.Balanced, TimeInterval: 5000,
distanceInterval: 10 },
    (loc) => setState((s) => ({ ...s, coords: toCoords(loc), error: null
}))
  )
  .then((sub) => {
    if (cancelled) sub.remove(); // se desmontó antes de resolver
    else subscription = sub;
  })

```

```

        .catch((e) =>
            setState((s) => ({ ...s, error: e instanceof Error ? e.message :
'Error de GPS' })))
        );

    return () => {
        cancelled = true;
        subscription?.remove();
    };
}, [watch, state.permission]);

const openSettings = useCallback(() => Linking.openSettings(), []);

return { ...state, requestPermission, getCurrent, openSettings };
}

```

Fíjate en tres cosas: la bandera `cancelled` resuelve la condición de carrera; el efecto de seguimiento depende de `state.permission` y arranca solo cuando el usuario concede el permiso; y los errores se exponen como estado para la UI, no como `console.log`.

Paso 4. Componente PermissionPrimer

```

// components/PermissionPrimer.tsx
import { View, Text, Pressable } from 'react-native';
import type { PermissionState } from '@types/geo';

interface Props {
    title: string;
    description: string;
    state: PermissionState;
    onRequest: () => void;
    onOpenSettings: () => void;
}

export function PermissionPrimer({ title, description, state, onRequest,
onOpenSettings }: Props) {
    const isBlocked = state === 'blocked';

    return (
        <View className="flex-1 items-center justify-center gap-4 bg-neutral-950
px-8">
            <Text className="text-center text-2xl font-bold text-white">{title}
</Text>
            <Text className="text-center text-base text-neutral-300">
                {isBlocked
                    ? 'Desactivaste este permiso. Puedes habilitarlo desde los Ajustes

```

```

del sistema.'
      : description}
    </Text>
    <Pressable
      onPress={isBlocked ? onOpenSettings : onRequest}
      className="rounded-full bg-emerald-500 px-6 py-3 active:opacity-80"
      accessibilityRole="button"
    >
      <Text className="font-semibold text-neutral-950">
        {isBlocked ? 'Abrir Ajustes' : 'Permitir acceso'}
      </Text>
    </Pressable>
  </View>
);
}

```

Paso 5. Custom Hook useCamera

```

// hooks/useCamera.ts
import { useCallback, useRef, useState } from 'react';
import { Linking } from 'react-native';
import { CameraView, useCameraPermissions, type CameraType } from 'expo-camera';
import type { PermissionState } from '@types/geo';

export function useCamera() {
  const cameraRef = useRef<CameraView>(null);
  const [permission, requestPermission] = useCameraPermissions();
  const [facing, setFacing] = useState<CameraType>('back');
  const [isReady, setIsReady] = useState(false);
  const [isCapturing, setIsCapturing] = useState(false);

  const permissionState: PermissionState = !permission
    ? 'checking'
    : permission.granted
    ? 'granted'
    : !permission.canAskAgain
    ? 'blocked'
    : permission.status === 'undetermined'
    ? 'undetermined'
    : 'denied';

  const toggleFacing = useCallback(() => {
    setFacing((f) => (f === 'back' ? 'front' : 'back'));
  }, []);
}

```

```

const takePhoto = useCallback(async () => {
  if (!cameraRef.current || !isReady || isCapturing) return null;
  setIsCapturing(true);
  try {
    const photo = await cameraRef.current.takePictureAsync({ quality: 0.7
  });
    return photo ?? null;
  } finally {
    setIsCapturing(false);
  }
}, [isReady, isCapturing]);

return {
  cameraRef,
  permissionState,
  requestPermission,
  openSettings: () => Linking.openSettings(),
  facing,
  toggleFacing,
  onCameraReady: () => setIsReady(true),
  takePhoto,
  isCapturing,
};
}

```

La guarda `isCapturing` evita dos capturas simultáneas por doble toque, que en algunos Android hacen fallar la cámara.

Paso 6. Pantalla GeoCam

```

// app/(tabs)/geocam.tsx
import { useState } from 'react';
import { View, Text, Pressable, Image, StyleSheet } from 'react-native';
import { CameraView } from 'expo-camera';
import { useCamera } from '@hooks/useCamera';
import { useGeoLocation } from '@hooks/useGeoLocation';
import { PermissionPrimer } from '@components/PermissionPrimer';
import type { GeoPhoto } from '@types/geo';

export default function GeoCamScreen() {
  const cam = useCamera();
  const geo = useGeoLocation({ watch: true });
  const [lastPhoto, setLastPhoto] = useState<GeoPhoto | null>(null);

  if (cam.permissionState === 'checking') {
    return <View className="flex-1 bg-neutral-950" />;
  }

```

```

}

if (cam.permissionState !== 'granted') {
  return (
    <PermissionPrimer
      title="GeoCam necesita tu cámara"
      description="La usamos solo para tomar fotos que tú decidas guardar."
      state={cam.permissionState}
      onRequest={cam.requestPermission}
      onOpenSettings={cam.openSettings}
    />
  );
}

const handleCapture = async () => {
  const photo = await cam.takePhoto();
  if (!photo) return;
  // Degradación elegante: sin permiso de ubicación, coords queda en null
  const coords =
    geo.permission === 'granted' ? geo.coords ?? (await geo.getCurrent()) :
null;

  setLastPhoto({
    id: String(Date.now()),
    uri: photo.uri,
    coords,
    source: 'camera',
    createdAt: Date.now(),
  });
};

return (
  <View className="flex-1 bg-black">
    /* CameraView sin hijos: los controles van como hermanos absolutos */
    <CameraView
      ref={cam.cameraRef}
      style={StyleSheet.absoluteFill}
      facing={cam.facing}
      onCameraReady={cam.onCameraReady}
    />

    /* Banner de ubicación: pedir en contexto sin bloquear la cámara */
    {geo.permission !== 'granted' && geo.permission !== 'checking' && (
      <Pressable
        onPress={geo.permission === 'blocked' ? geo.openSettings :
geo.requestPermission}

```

```

        className="absolute left-4 right-4 top-14 rounded-xl bg-amber-
400/90 p-3"
    >
        <Text className="text-center font-medium text-neutral-900">
            Activa la ubicación para etiquetar tus fotos
        </Text>
    </Pressable>
})

{/* Coordenadas en vivo */}
{geo.coords && (
    <View className="absolute left-4 top-14 rounded-lg bg-black/60 px-3
py-2">
        <Text className="font-mono text-xs text-emerald-300">
            {geo.coords.latitude.toFixed(5)},
{geo.coords.longitude.toFixed(5)}
        </Text>
        <Text className="font-mono text-xs text-neutral-400">
            ±{Math.round(geo.coords.accuracy ?? 0)} m
        </Text>
    </View>
)}

{/* Controles inferiores */}
<View className="absolute bottom-10 left-0 right-0 flex-row items-
center justify-around">
    {lastPhoto ? (
        <Image source={{ uri: lastPhoto.uri }} className="h-14 w-14
rounded-lg border-2 border-white" />
    ) : (
        <View className="h-14 w-14" />
    )}

    <Pressable
        onPress={handleCapture}
        disabled={cam.isCapturing}
        className="h-20 w-20 items-center justify-center rounded-full
border-4 border-white active:opacity-70"
        accessibilityLabel="Tomar foto"
    >
        <View className={`h-16 w-16 rounded-full ${cam.isCapturing ? 'bg-
neutral-400' : 'bg-white'} `} />
    </Pressable>

    <Pressable
        onPress={cam.toggleFacing}

```

```

        className="h-14 w-14 items-center justify-center"
        accessibilityLabel="Cambiar cámara"
      >
        <Text className="text-3xl">v</Text>
      </Pressable>
    </View>
  </View>
);
}

```

Paso 7. Verifica en tu teléfono

- ☐ Niega la ubicación y verifica que la cámara sigue funcionando.
- ☐ Niega la cámara dos veces en Android hasta llegar a `blocked` y comprueba que aparece "Abrir Ajustes".
- ☐ Sal de la pestaña y confirma con un log temporal en el cleanup que el GPS se detiene.

Escribir coordenadas dentro del EXIF del JPG requiere librerías nativas fuera de Expo Go. Por eso la foto y sus coordenadas se guardan como un objeto `GeoPhoto`, que será el primer esquema de SQLite en la Semana 7.

Taller en clase: GeoCam completa

En parejas, extiende la GeoCam de la práctica guiada a una app de dos pestañas sobre el repositorio del Taller Integrador 2. Puedes usar IA libremente, registrando todo en `AI-LOG.md`.

Requisitos

- ☐ **R1. Estado global de fotos.** Crea `GeoPhotosContext` con `addPhoto`, `removePhoto` y `clearAll`, con Provider en el `_layout.tsx` de tabs y actualizaciones inmutables (Semana 4). GeoCam usa `addPhoto` en lugar del estado local `lastPhoto`.
- ☐ **R2. Importar desde galería.** Botón con `expo-image-picker`; las fotos se guardan con `source: 'gallery'`, con las coordenadas actuales o `coords: null`, y se distinguen visualmente de las de cámara.
- ☐ **R3. Pestaña Mapa.** `app/(tabs)/mapa.tsx` con un `Marker` por foto con coordenadas; al tocarlo se ve la miniatura. Las fotos sin coordenadas se listan aparte como "Sin ubicación". El mapa se centra en tu ubicación o, sin permiso, en la última foto.
- ☐ **R4. Custom Hook useShake.** Al agitar el teléfono, un `Alert` pregunta si se borran todas las fotos.

- ☐ **R5. AI-LOG.** Al menos un prompt, una corrección y una alucinación detectada.

Código base para R2:

```
const result = await ImagePicker.launchImageLibraryAsync({
  mediaTypes: ['images'],
  quality: 0.7,
});
if (!result.canceled) {
  // result.assets[0].uri -> crear GeoPhoto con source: 'gallery'
}
```

Requisitos técnicos de R4:

1. Verificar `Accelerometer.isAvailableAsync()` antes de suscribirse.
2. Fijar `Accelerometer.setUpdateInterval(100)`.
3. Aplicar un umbral sobre la magnitud del vector: $\sqrt{x^2 + y^2 + z^2}$, que en reposo vale cerca de 1 g.
4. Incluir un tiempo de enfriamiento para no disparar varias veces por un solo agitado.
5. Limpiar la suscripción con la bandera `cancelled`, igual que en `useGeolocation`.

```
export function useShake(
  onShake: () => void,
  options?: { threshold?: number; cooldownMs?: number }
): { isAvailable: boolean | null };
```

Retos opcionales

- **Nivel con giroscopio:** en GeoCam, un indicador de nivel que ayude a tomar la foto derecha, con `Gyroscope` o `DeviceMotion`.
- **EXIF de galería:** con `exif: true` en el picker, lee las coordenadas originales de fotos importadas. Las claves y el formato GPS difieren entre Android e iOS; documenta en el AI-LOG qué devolvió cada plataforma.
- **Ubicación aproximada:** si la precisión supera 1 km, avisa al usuario que concedió ubicación aproximada.

Errores de IA que debes detectar

Las APIs de Expo cambiaron mucho entre SDKs y los modelos mezclan versiones. Si aparece alguno de estos casos, repórtalo en tu AI-LOG:

La IA genera	Problema	Corrección
<code>import { Camera } from 'expo-camera' con <code>Camera.requestCameraPermissionsAsync()</code></code>	API antigua, reemplazada	<code>CameraView</code> + <code>useCameraPermissions()</code>
<code>import * as Permissions from 'expo-permissions'</code>	Paquete obsoleto y retirado	Permisos desde cada módulo, p. ej. <code>Location.requestForegroundPermissionsAsync()</code>
<code>mediaTypes: ImagePicker.MediaTypeOptions.Images</code>	Opción obsoleta	<code>mediaTypes: ['images']</code>
Controles como hijos de <code><CameraView></code>	No soportado, comportamiento inconsistente	Hermanos con posición absoluta
<code>watchPositionAsync</code> o <code>addListener</code> sin <code>.remove()</code>	Fuga de batería y memoria	Cleanup en <code>useEffect</code> con bandera <code>cancelled</code>
Pedir todos los permisos al iniciar la app	Mala UX; en iOS se pierde la única oportunidad	Pedir en contexto con pantalla previa
<code>npm install expo-camera</code>	Posible versión incompatible con el SDK	<code>npx expo install expo-camera</code>
<code>requestBackgroundPermissionsAsync</code> para etiquetar fotos	Permiso excesivo, rechazo en tiendas	Solo <code>foreground</code>

Entregable semanal

Entrega el enlace a tu repositorio de GitHub con al menos un Custom Hook nativo con manejo de permisos funcional.

Qué debe contener tu repositorio

- ☐ `hooks/useGeoLocation.ts` o `hooks/useCamera.ts` (idealmente ambos) y `hooks/useShake.ts`.
- ☐ `README.md` con capturas o GIF de los tres estados de permiso: concedido, rechazado y bloqueado.
- ☐ `AI-LOG.md` de la Semana 6 en la raíz.

Antes de entregar, verifica

- ☐ La app pide permisos en contexto, nunca al abrirse.
- ☐ Si niegas la ubicación, la cámara sigue funcionando y la app lo comunica.
- ☐ Existe un botón "Abrir Ajustes" cuando el permiso está bloqueado.
- ☐ Todas las suscripciones (GPS, acelerómetro) se limpian al salir de la pantalla.
- ☐ Los hooks están tipados, sin `any`, y los errores llegan a la UI como estado.

Ejemplo de AI-LOG.md

```
# Registro de Auditoría de IA (AI-LOG)

**Estudiante(s):** Nombre Apellido, Nombre Apellido
**Semana:** 6
**Proyecto:** GeoCam - Taller Integrador 2

## 1. Prompts Utilizados
- "Escribe un Custom Hook en React Native con TypeScript que detecte cuando el usuario agita el teléfono usando expo-sensors, con umbral configurable."

## 2. Código Generado vs. Código Modificado
- **¿Qué generó la IA?:** Un hook que se suscribía a Accelerometer dentro de useEffect, sin setInterval y disparando el callback decenas de veces por cada agitado.
- **¿Qué modifiqué/corregí?:** Agregué setInterval(100), un cooldown de 1000 ms y la verificación con isAvailableAsync. Guardé el callback en una ref para no reiniciar la suscripción en cada render.

## 3. Alucinaciones o Errores Detectados
- La IA usó `import { Camera } from 'expo-camera'` y `Camera.requestCameraPermissionsAsync()`, una API antigua. La reemplacé por `CameraView` y el hook `useCameraPermissions()` según la documentación oficial del SDK actual.
```

Documentación de referencia

- [Expo Camera](#)
- [Expo Location](#)
- [Expo ImagePicker](#)
- [Expo Sensors](#)
- [Guía de permisos en Expo](#)
- [React Native Maps](#)

Consejo: revisa la versión del SDK de Expo de tu proyecto (campo `expo` en `package.json`) y consulta la documentación de esa versión. Si la IA te propone un nombre que no aparece ahí, probablemente es una alucinación.